

ShealtRI–YourHealthWiki: un Sistema de Recuperación de Información Médica basado en Indexado Semántico Latente y Generación Aumentada por Recuperación

Lianny de la C. Reveé Valdivieso¹, Kevin A. Torres Perera¹, and Jocdan L. López Mantecón¹

¹ Ciencias de la Computación, Grupos 312, Universidad de La Habana

² Ciencia de Datos, Grupos 311 y 312, Universidad de La Habana

<https://github.com/MatCom-Developing-Team-10/ShealtRI-YourHealthWiki.git>

Abstract This paper presents *ShealtRI–YourHealthWiki*, an information retrieval (IR) system for the health and medicine domain that lets users query medical information in natural language and returns relevant documents together with a profile-aware, retrieval-augmented answer. The non-basic retrieval model is Latent Semantic Indexing (LSI): a TF–IDF term–document matrix is reduced with truncated singular value decomposition (SVD) to a latent space of $k = 100$ dimensions, and the resulting latent vectors are used as embeddings inside a ChromaDB vector store searched by cosine similarity. The architecture is a Pipeline+Microkernel modular monolith: a fixed retrieval pipeline (parsing, spell correction, retrieval, hybrid re-ranking, generation) is extended by optional plugins (thesaurus-based query expansion, Rocchio relevance feedback, a content-based recommender, and an evaluation module) that attach through declarative hooks without contaminating the mandatory code. A sitemap-driven crawler acquires curated content from MedlinePlus, Mayo Clinic and the NHS while honouring `robots.txt`, per-domain rate limiting and retry policies. When the local corpus is insufficient—detected through result-count, score and recency criteria—an automatic DuckDuckGo web fallback supplements and is cached for reuse. Retrieved documents are re-ranked by a hybrid LSI+BM25 scorer and passed to a Groq-hosted large language model grounded strictly on the retrieved context to mitigate hallucination, with a template fallback when no model is available. We describe the end-to-end data flow, the two-level storage design, the dynamic-indexing (folding-in) mechanism, and an evaluation module reporting Precision, Recall, F1, NDCG, MAP and MRR. We close with the detected limitations and the design we would pursue under idealised conditions.

Keywords: recuperación de información, indexado semántico latente, LSI, TF–IDF, RAG, retroalimentación de Rocchio, dominio médico

Índice general

ShealtRI–YourHealthWiki	1
<i>Lianny de la C. Reveé Valdivieso, Kevin A. Torres Perera, and Jocdan L. López Mantecón</i>	
1. Introducción	3
2. Marco Teórico	3
2.1. El modelo formal de un SRI	3
2.2. Modelo de espacio vectorial y ponderación TF–IDF	3
2.3. Indexado Semántico Latente (LSI): el modelo no básico	4
2.4. Re-ordenamiento híbrido, BM25 y RAG	4
2.5. Evaluación	5
3. Desarrollo e Implementación	5
3.1. Arquitectura general	5
3.2. Adquisición de datos: Crawler	5
3.3. El corpus y su representatividad	6
3.4. Indexación: índice invertido y normalización	7
3.5. Modelo LSI, embeddings y base de datos vectorial	7
3.6. Flujo <i>end-to-end</i> de una consulta	8
3.7. Detección de información insuficiente y búsqueda web	9
3.8. Re-ranking, factores de ordenamiento y posicionamiento visual ..	9
3.9. Generación aumentada por recuperación (RAG)	9
3.10. Plugins opcionales: expansión, retroalimentación y recomendación	10
4. Evaluación y Resultados	11
5. Deficiencias, Conclusiones y Trabajo Futuro	11

1. Introducción

Los Sistemas de Recuperación de Información (SRI) constituyen la disciplina que estudia la representación, el almacenamiento, la organización y el acceso a ítems de información para satisfacer una necesidad expresada por un usuario [1,2]. Su relevancia es hoy mayor que nunca: el volumen de texto disponible crece a un ritmo que vuelve impracticable la búsqueda puramente manual, y la calidad de las decisiones —especialmente en dominios sensibles como la salud— depende de poder localizar, en segundos, contenido fiable y pertinente. A diferencia de un sistema de bases de datos, que devuelve coincidencias exactas sobre datos estructurados, un SRI opera sobre lenguaje natural intrínsecamente ambiguo y debe ordenar los resultados por relevancia estimada, no por igualdad.

Este informe documenta *ShealtRI–YourHealthWiki*, un SRI para el dominio de **salud y medicina** desarrollado como proyecto integrador. El sistema permite formular consultas en lenguaje natural sobre síntomas, enfermedades, medicamentos y cuidados, y recupera documentos médicos relevantes empleando *Latent Semantic Indexing* (LSI) como modelo de recuperación no básico, enriqueciendo la respuesta mediante *Retrieval-Augmented Generation* (RAG) adaptada al perfil del usuario. Por un lado, ofrecer una experiencia de búsqueda médica que tolere errores ortográficos y sinonimia —frecuentes cuando un paciente describe su problema con vocabulario no técnico—; por otro, garantizar que la respuesta generada esté *anclada* en fuentes recuperadas verificables, evitando la fabricación de información.

El alcance abarca todo el ciclo de un SRI: adquisición de datos mediante un *crawler* respetuoso de las políticas de los sitios, indexación con índice invertido y reducción semántica, recuperación y re-ordenamiento híbrido, generación de respuestas, y un módulo de evaluación cuantitativa.

2. Marco Teórico

2.1. El modelo formal de un SRI

Siguiendo la formalización clásica [3,2], un SRI se define como una cuádrupla $\langle D, Q, F, R \rangle$. En ShealtRI, D es la colección de documentos médicos representados como vectores en un espacio semántico latente $\mathbb{R}^k$; Q es el conjunto de consultas en lenguaje natural, normalizadas, corregidas ortográficamente y proyectadas al mismo espacio; F es el *framework* de emparejamiento, esto es, el modelo de espacio vectorial latente con similitud del coseno, complementado por una fusión léxica con BM25; y R es la función de ranking $R(q, d) \in [0, 1]$ que ordena los documentos por relevancia estimada para cada consulta.

2.2. Modelo de espacio vectorial y ponderación TF–IDF

El modelo de espacio vectorial representa documentos y consultas como vectores sobre el vocabulario del corpus [3]. La componente asociada a un término

t en un documento d se pondera con el esquema TF-IDF,

$$w_{t,d} = \text{tf}_{t,d} \cdot \log \frac{N}{\text{df}_t}, \quad (1)$$

donde $\text{tf}_{t,d}$ es la frecuencia del término en el documento, N el número de documentos y df_t la frecuencia documental de t . TF-IDF premia los términos frecuentes en un documento pero raros en la colección, que son los más discriminantes [1]. La relevancia entre consulta q y documento d se mide con la similitud del coseno, $\text{sim}(q, d) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$.

2.3. Indexado Semántico Latente (LSI): el modelo no básico

El modelo de espacio vectorial puro sufre dos problemas léxicos bien conocidos: la *sinonimia* (términos distintos con igual significado, p.ej. “infarto” y “ataque al corazón”) y la *polisemia*. LSI [4] los mitiga aplicando una descomposición en valores singulares truncada (truncated SVD) sobre la matriz término-documento A (de TF-IDF):

$$A \approx U_k \Sigma_k V_k^\top, \quad (2)$$

donde U_k , Σ_k y $V_k^\top$ retienen las k mayores componentes. Las columnas de $V_k^\top$ son las representaciones de los documentos en un espacio *latente* de k dimensiones que captura relaciones de co-ocurrencia entre términos, de modo que documentos semánticamente afines quedan cercanos aunque no compartan vocabulario exacto. Una consulta se proyecta al mismo espacio mediante $\mathbf{q}_{\text{proj}} = \mathbf{q}^\top U_k \Sigma_k^{-1}$, y luego se comparan los vectores por coseno. La ventaja frente a los modelos básicos (booleano y vectorial puro) para el dominio médico es directa: mejora el *recall* ante la sinonimia abundante de la terminología clínica y reduce el ruido, a la vez que produce vectores densos de baja dimensión que se almacenan e indexan eficientemente en una base vectorial. La implementación se apoya en la formulación de [4,1] y en `scikit-learn` (TruncatedSVD sobre `TfidfVectorizer`).

2.4. Re-ordenamiento híbrido, BM25 y RAG

BM25 [5] es una función de ranking probabilística que puntúa el solapamiento léxico exacto entre consulta y documento con saturación de frecuencia y normalización por longitud. Combinar una señal densa (LSI, orientada a *recall* semántico) con una señal dispersa (BM25, orientada a la *precisión* del término exacto) es la técnica estándar de recuperación híbrida. La Generación Aumentada por Recuperación (RAG) [8] antepone una etapa de recuperación a un modelo generativo: el modelo redacta su respuesta condicionado a los documentos recuperados, lo que reduce las alucinaciones y permite citar fuentes. La retroalimentación de relevancia de Rocchio [6] re-pondera el vector de consulta acercándolo al centroide de los documentos marcados como relevantes y alejándolo de los no relevantes. Para diversificar recomendaciones se emplea *Maximal Marginal Relevance* (MMR) [7].

2.5. Evaluación

La calidad se mide con las medidas objetivas clásicas [1]: Precisión $P = |RR|/|RR \cup RI|$, Recobrado $R = |RR|/|RR \cup NR|$ y su media armónica $F_1 = 2PR/(P + R)$, donde RR , RI y NR son, respectivamente, los documentos relevantes recuperados, los irrelevantes recuperados y los relevantes no recuperados. Como P , R y F_1 ignoran el orden, se añaden medidas sensibles al ranking: la ganancia acumulada descontada normalizada $NDCG@k = DCG@k/IDCG@k$ con $DCG@k = \sum_{i=1}^k rel_i / \log_2(i + 1)$ [9], la precisión media (MAP) y el recíproco del rango medio (MRR).

3. Desarrollo e Implementación

3.1. Arquitectura general

El sistema se implementa en Python como un **monolito modular** que combina los patrones *Pipeline* y *Microkernel* (Fig. 1). El flujo de una consulta es un *pipeline* de etapas fijas y obligatorias; los módulos opcionales se conectan como *plugins* a través de *hooks* declarativos (`pre_retrieval`, `post_retrieval`, `post_ranking`) gestionados por `core/plugin_pipeline.py`. Esta decisión arquitectónica —frente a microservicios o *Clean Architecture* pura, descartados por su sobrecarga para un equipo pequeño— aporta tres beneficios: los *plugins* no contaminan el código obligatorio (si un *plugin* no se registra, el *pipeline* simplemente lo ignora), cada módulo implementa una interfaz abstracta (`core/interfaces.py`) y es testeable de forma aislada, y todo corre en un único proceso, reproducible con Docker.

3.2. Adquisición de datos: Crawler

La adquisición la realiza un *crawler* genérico (`modules/crawler`) que separa estrictamente la lógica de E/S de la extracción específica de cada sitio. Las fuentes elegidas son tres servicios de salud de reputación contrastada: **MedlinePlus** (Biblioteca Nacional de Medicina de EE. UU., con contenido en español e inglés), **Mayo Clinic** y el **NHS** del Reino Unido. La confiabilidad y actualidad se aseguran por la naturaleza institucional y curada de estas fuentes y porque cada documento conserva en sus metadatos la fecha de última modificación declarada por el sitio.

El descubrimiento de URLs es *exclusivamente por sitemaps*: el *crawler* obtiene los `sitemap.xml` declarados por cada *scraper*, expande recursivamente los `sitemapindex` (hasta una profundidad máxima de 3) y construye una cola de pares (URL, *scraper*). Las semillas, por tanto, son los *sitemaps* de los tres dominios. *La búsqueda no sale del dominio del conjunto semilla*: al no implementarse seguimiento de enlaces y al filtrar cada URL con `can_handle()` contra el dominio del *scraper*, el alcance queda acotado a las fuentes curadas, lo que es deseable para mantener la calidad del corpus. El respeto de las políticas

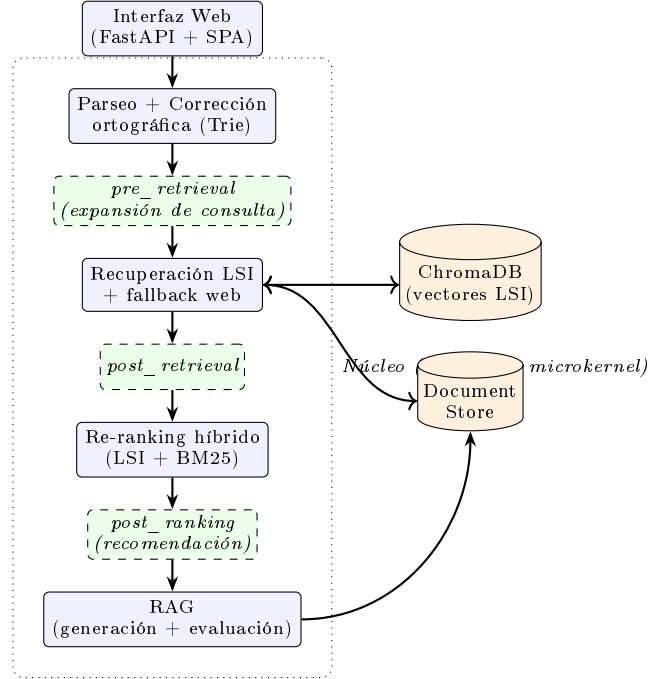


Figura 1. Arquitectura Pipeline+Microkernel. Las etapas en azul son obligatorias; los recuadros verdes punteados son *hooks* donde se enganchan plugins opcionales; los cilindros son los dos niveles de almacenamiento.

de *crawling* es explícito: se consulta y cachea el `robots.txt` de cada dominio (`RobotFileParser.can_fetch`, con política *fail-open*), se impone un *rate limiting* por dominio (retardo mínimo configurable de 2 s entre peticiones), se aplica una política de reintentos con *backoff* exponencial ante códigos 429/5xx, un *timeout* de 15 s, un *User-Agent* identificable (MedSRIBot/1.0 (proyecto académico UH)) y un límite opcional `max_pages`. Cada documento recibe un identificador estable UUID5 derivado de su URL, lo que hace la adquisición idempotente entre re-crawls.

3.3. El corpus y su representatividad

El corpus indexado es *textual*. Se compone de los documentos JSONL producidos por el crawler en `data/raw/` y de capítulos de libros médicos preprocesados a JSON (ingeridos página a página, con identificadores del tipo `<libro>_p<NNN>`). La suficiencia y representatividad se persiguen por construcción: combinar tres fuentes institucionales complementarias (enciclopedia médica, contenido para pacientes y guías clínicas) cubre tanto el registro divulgativo como el técnico, y el carácter bilingüe (es/en) amplía la cobertura terminológica. El recuento exacto de documentos depende de la ejecución del crawler; el

sistema reporta en todo momento, mediante el endpoint `/api/stats`, el número de documentos originales, de *chunks*, el tamaño del vocabulario y la dimensión latente k efectivamente usada.

3.4. Indexación: índice invertido y normalización

La indexación (`modules/indexer`) construye un **índice invertido** que mapea cada término a una lista de *postings* (`idx_doc, tf`). Antes, cada documento atraviesa el `TextProcessor` (`modules/text_processor`), que aplica: normalización Unicode y a minúsculas, eliminación de caracteres no alfanuméricos preservando los propios del español (áéíóúüñ), tokenización y lematización con `spaCy` (`es_core_news_md`), eliminación de *stopwords* (NLTK español más una lista de *stopwords* de dominio médico) y filtrado por longitud (2–20 caracteres). Los documentos largos se fragmentan previamente con un `TextChunker` (ventanas fijas de 300 palabras con solape de 50), de modo que cada *chunk* es una unidad recuperable independiente que conserva en sus metadatos el `original_doc_id`. El vocabulario se ordena alfabéticamente (columnas deterministas) y se descartan los términos con frecuencia total inferior a 2, filtrando ruido y erratas. Como subproducto, los tokens lematizados de los documentos alimentan el vocabulario del corrector ortográfico (un **Trie** con distancia de Levenshtein) que más tarde corrige las consultas.

3.5. Modelo LSI, embeddings y base de datos vectorial

Sobre el índice invertido, `TfidfProcessor` construye la matriz TF-IDF y `LSIModel` (envoltorio de `TruncatedSVD`, $k = 100$, `n_iter=7`, semilla fija) la reduce al espacio latente. **Los vectores latentes resultantes son los *embeddings* del sistema**: no se emplea un modelo de *embeddings* neuronal externo, sino la propia proyección LSI, coherente con el modelo de recuperación elegido y reproducible sin dependencias de servicios en la nube. Estos vectores se almacenan en **ChromaDB**, configurado con espacio coseno (`hnsw:space=cosine`); la búsqueda por similitud usa el índice aproximado HNSW de ChromaDB, eficiente para hallar los vecinos más cercanos en alta dimensión sin recorrer toda la colección.

El almacenamiento es de **dos niveles**: ChromaDB guarda únicamente IDs, vectores y metadatos mínimos (URL), mientras que el contenido completo reside en un `FileSystemDocumentStore`. Esta separación evita inflar la base vectorial, permite actualizar contenido sin re-indexar vectores y optimiza cada almacén para su propósito. Para información nueva, el sistema implementa *indexación dinámica* (*folding-in*, Fig. 2): un documento nuevo se vectoriza con la TF-IDF e IDF ya fijadas y se proyecta sobre la base latente existente sin reentrenar la SVD ($\mathbf{d}_{\text{proj}} = \mathbf{d} V_k$), y luego se anexa a ambos almacenes. Cuando la fracción de documentos añadidos por *folding-in* supera el 20 % se dispara un re-ajuste completo (“balanceo”) que recaptura el vocabulario nuevo.

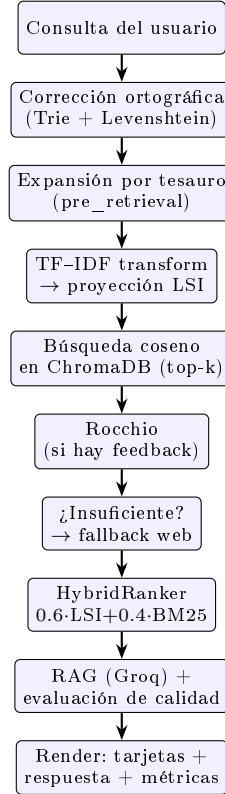


Figura 2. Flujo *end-to-end* de una consulta y módulos activados en orden.

3.6. Flujo *end-to-end* de una consulta

La Fig. 2 narra el recorrido completo. (1) El usuario escribe en la interfaz web y la petición llega a `POST /api/query`. (2) `Indexer.build_query` normaliza y *corrige ortográficamente* la consulta contra el vocabulario del Trie; un mecanismo de interfaz permite además elegir entre la consulta corregida o la original tal cual se escribió. (3) En el *hook pre_retrieval*, el plugin de expansión enriquece la consulta con sinónimos médicos. (4) El `LSIRetriever` vectoriza con TF-IDF (descartando términos fuera de vocabulario), proyecta al espacio latente y realiza la *recuperación en dos fases*: búsqueda vectorial en ChromaDB → IDs+scores, y recuperación del texto completo en el Document Store. (5) Si existen juicios de relevancia previos para esa consulta, se aplica Rocchio sobre el vector latente. (6) El `FallbackRetriever` decide si activar la búsqueda web. (7) Tras *post_retrieval*, el `HybridRanker` re-ordena por puntuación híbrida. (8) En *post_ranking* se preparan las recomendaciones. (9) El `RAGService` genera la respuesta y el `RAGEvaluator` la puntúa. El resultado (documentos, respuesta, métricas y sugerencias) se serializa y se renderiza.

3.7. Detección de información insuficiente y búsqueda web

El `FallbackRetriever` decide que “lo indexado no satisface la necesidad” mediante tres criterios combinados: (i) *conteo*, cuando la recuperación local devuelve menos de `min_results=3` documentos; (ii) *puntuación*, cuando el mejor resultado local queda por debajo de `min_score=0.35`; y (iii) *recencia*, cuando la consulta contiene marcas temporales (años 2020–2029, “última hora”, “nuevo tratamiento”, “actualizado”, etc.), pues el corpus curado puede no reflejar lo más reciente. Al activarse, el `InternetSearchRetriever` consulta DuckDuckGo (biblioteca `ddgs`), descarga y limpia las páginas con `WebContentFetcher` (`requests+BeautifulSoup`), asigna puntuaciones decrecientes por posición y marca los documentos con `origin="web"`. Los resultados web se *fusionan* con los locales deduplicando por `doc_id` (conservando la mayor puntuación) y se ordenan por `score`; además se *cachean* en el Document Store para reutilizarse en consultas idénticas posteriores, evitando nuevas llamadas de red.

3.8. Re-ranking, factores de ordenamiento y posicionamiento visual

El `HybridRanker` produce el orden final fusionando dos señales normalizadas a $[0, 1]$ por min-max: la similitud latente LSI (peso 0.6) y la puntuación léxica BM25 (peso 0.4), de modo que ningún criterio domine por escala. Además de la relevancia, intervienen otros factores: la *frescura* (consultas marcadas como recientes desvían hacia resultados web actuales), el *tipo/origen de contenido* (local frente a web, señalizado con insignias) y el *perfil del usuario*, que el recomendador traduce en un *boost* multiplicativo conservador (1.05–1.10) hacia las fuentes preferidas de cada perfil (p. ej. MedlinePlus para pacientes y cuidadores; Mayo Clinic/NHS para profesionales). Dos consultas equivalentes con perfiles distintos, o una consulta con marca de recencia frente a otra sin ella, producen así ordenamientos observablemente diferentes.

En la interfaz (Fig. 3), el posicionamiento responde a estas decisiones de diseño: los documentos se presentan como tarjetas ordenadas de arriba a abajo por puntuación híbrida descendente —el primero es el más relevante—, cada tarjeta muestra título, fuente enlazable, un *snippet* sensible a la consulta (la ventana de texto con mayor densidad de términos buscados) y un indicador porcentual de relevancia; una insignia distingue visualmente el contenido *Local* del *Web*. La respuesta generada por RAG ocupa un panel lateral propio para separar claramente “documentos fuente” de “respuesta sintetizada”, y un panel inferior de recomendaciones (“También te puede interesar”) agrupa contenido nuevo relacionado sin interferir con el ranking principal.

3.9. Generación aumentada por recuperación (RAG)

El `RAGService` produce una respuesta adaptada a uno de seis perfiles de usuario (paciente, estudiante de medicina, profesional, asistente de diagnóstico, medicina natural y cuidador), cada uno con su propia plantilla de *system*

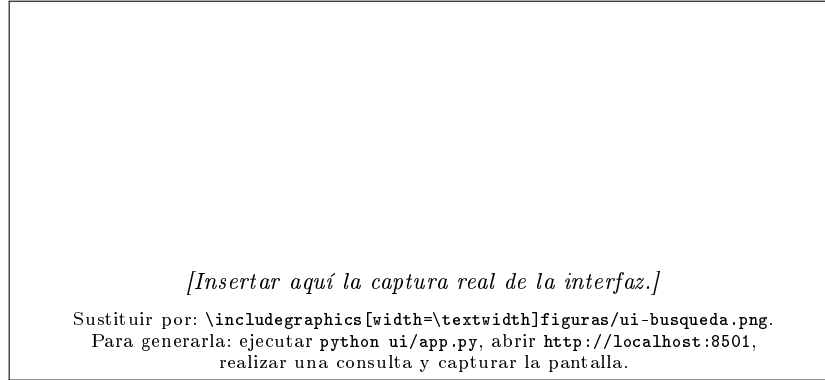


Figura 3. Interfaz de ShealtRI: barra de búsqueda con conmutadores (corrección ortográfica e inclusión de resultados web), tarjetas de resultados ordenadas por relevancia con insignias de origen, y panel lateral con la respuesta RAG.

prompt, tono y áreas de enfoque. El proceso es: (1) se re-ordenan los documentos con el `HybridRanker`; (2) se construye un *bloque de contexto* con a lo sumo los **3 documentos** mejor posicionados, recortando cada uno a 800 caracteres —es decir, *no* se pasa todo lo recuperado, sino lo priorizado por relevancia—; (3) se renderiza el *prompt* insertando consulta y contexto; (4) se invoca el modelo alojado en Groq (`llama-3.1-8b-instant`, `temperature=0.3`, `top_p=0.9`, `max_tokens=1024`). La **mitigación de alucinaciones** es estructural: el *prompt* ancla explícitamente la respuesta a la sección “Información de referencia: {context}” e instruye al modelo a *indicar expresamente cuando los documentos de referencia son insuficientes*; la baja temperatura reduce la divagación. Si no hay clave de API o la llamada falla, el sistema degrada con elegancia a una *respuesta de plantilla* que presenta directamente los *snippets* recuperados. Por último, un `RAGEvaluator` puntúa cada respuesta en fidelidad, anclaje (*groundedness*) y relevancia del contexto, métricas que la interfaz expone como indicador de calidad.

3.10. Plugins opcionales: expansión, retroalimentación y recomendación

Coherentes con el microkernel, los módulos opcionales se enganchan sin tocar el núcleo. La **expansión de consultas** (`plugins/expansion`, `hook pre_retrieval`) busca cada término en un tesoro médico y añade sus sinónimos, abreviaturas y formas coloquiales al índice invertido de la consulta, filtrando los que no estén en el vocabulario del corpus y limitando a 6 términos para evitar la *deriva de consulta*. La **retroalimentación de relevancia** (`plugins/feedback`) persiste los juicios del usuario (pulgar arriba/abajo en cada tarjeta) y, en la siguiente búsqueda de la misma consulta, aplica Rocchio ($\alpha = 1.0$, $\beta = 0.75$, $\gamma = 0.15$) sobre el vector latente, acercándolo al centroide de lo relevante. El **recomendador** (`modules/recommender`) parte de los primeros resultados como semilla,

Cuadro 1. Métricas implementadas en el módulo de evaluación y su rol.

Métrica	Sensible al orden	Qué mide
Precisión / Recobrado / F_1	No	Calidad del conjunto recuperado
$P@k$ / $R@k$ / $Fallout@k$	Parcial (corte k)	Calidad en el top- k
MAP	Sí	Precisión media sobre relevantes
MRR	Sí	Posición del primer acierto
NDCG@ k	Sí	Ganancia graduada con descuento

calcula un centroide en el espacio LSI, recupera candidatos por coseno y los reordena con MMR ($\lambda = 0.7$) para que la lista sea afín pero diversa, colapsando además páginas del mismo libro para no mostrar duplicados.

4. Evaluación y Resultados

El módulo de evaluación (`modules/evaluation`) implementa, como funciones puras y testeables, todas las medidas de la Sección 2: P , R , F_1 y F_β general, $P@k$, $R@k$, $Fallout@k$, MRR, MAP, DCG y NDCG@ k . Sobre un conjunto de 10 consultas de prueba de dominio biomédico (`eval_queries.json`) con sus juicios de relevancia (`eval_qrels.json`), el endpoint `/api/eval` ejecuta el recuperador LSI y reporta tanto las métricas agregadas como el desglose por consulta, mostrándolos en la sección “Evaluación” de la interfaz (Tabla 1). El recobrado del corpus completo (*Fallout*) se calcula contra el número de documentos *originales* —no de *chunks*— para que el denominador sea coherente. Los valores numéricos concretos dependen del corpus efectivamente crawlado en cada despliegue; la infraestructura de evaluación permite reproducir la medición con un solo comando (`python cli.py -eval -eval-k 10`) y comparar LSI puro contra LSI con re-ranking híbrido (`-rerank`).

5. Deficiencias, Conclusiones y Trabajo Futuro

ShealtRI demuestra que un SRI médico completo —adquisición, indexación semántica, recuperación híbrida, fallback web y RAG— puede construirse como un monolito modular reproducible, manteniendo cada responsabilidad aislada y testeable. No obstante, detectamos varias deficiencias y sus posibles soluciones. Primero, la base latente de LSI es *estática*: los documentos añadidos por *folding-in* no aportan vocabulario nuevo hasta el “balanceo”, y los documentos web recuperados se cachean pero no se integran al espacio latente en la misma consulta; un trabajo futuro sería re-ajustar la SVD de forma incremental o adoptar *embeddings* neuronales (*sentence-transformers*) que toleren vocabulario abierto. Segundo, el corpus puede quedar desactualizado frente a la evidencia médica reciente; integrar fuentes con *feeds* fechados y un re-crawl programado lo mitigaría. Tercero, el Trie del corrector no se contrae al eliminar documentos, por lo que puede sugerir términos ya no indexados. Cuarto, el conjunto de

evaluación es pequeño (10 consultas); ampliarlo con juicios de relevancia graduados por especialistas daría medidas más robustas. Finalmente, el sistema es solo textual: la recuperación multimodal de imágenes médicas queda como extensión.

Bajo condiciones idílicas y empezando de cero, mantendríamos el patrón Pipeline+Microkernel —que demostró su valor para entregas incrementales— pero adoptaríamos desde el inicio *embeddings* densos neuronales como representación primaria (con LSI como línea base comparativa), un índice vectorial con actualización verdaderamente incremental, y evaluación amplia y validada clínicamente. La separación de almacenamiento en dos niveles, la fusión híbrida y el anclaje estricto del RAG se conservarían sin cambios, por haber resultado decisiones acertadas.

Referencias

1. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, Cambridge (2008)
2. Baeza-Yates, R., Ribeiro-Neto, B.: Modern Information Retrieval. ACM Press / Addison-Wesley, New York (1999)
3. Salton, G., McGill, M.J.: Introduction to Modern Information Retrieval. McGraw-Hill, New York (1983)
4. Deerwester, S., Dumais, S.T., Furnas, G.W., Landauer, T.K., Harshman, R.: Indexing by Latent Semantic Analysis. J. Am. Soc. Inf. Sci. 41(6), 391–407 (1990)
5. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Found. Trends Inf. Retr. 3(4), 333–389 (2009)
6. Rocchio, J.J.: Relevance Feedback in Information Retrieval. In: Salton, G. (ed.) The SMART Retrieval System: Experiments in Automatic Document Processing, pp. 313–323. Prentice-Hall, Englewood Cliffs (1971)
7. Carbonell, J., Goldstein, J.: The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In: Proc. 21st ACM SIGIR, pp. 335–336. ACM, New York (1998)
8. Lewis, P., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474 (2020)
9. Järvelin, K., Kekäläinen, J.: Cumulated Gain-Based Evaluation of IR Techniques. ACM Trans. Inf. Syst. 20(4), 422–446 (2002)
10. Chroma: the open-source embedding database, <https://www.trychroma.com>
11. Honnibal, M., Montani, I.: spaCy: Industrial-Strength Natural Language Processing in Python, <https://spacy.io>
12. Pedregosa, F., et al.: Scikit-learn: Machine Learning in Python. J. Mach. Learn. Res. 12, 2825–2830 (2011)
13. U.S. National Library of Medicine: MedlinePlus, <https://medlineplus.gov>